AWS SAA（SAA-C03）　分野3 高パフォーマンス　#12

EC2

仮想サーバーを、用途に合った形で動かす。インスタンスタイプの選び方、起動と構成（AMI・ユーザーデータ・メタデータ・インスタンスストア・休止・自動復旧）、ネットワークと配置（ENI・Elastic IP・ENA・EFA・プレイスメントグループ）

各ページの下に、根拠にした AWS の公式ドキュメント（Amazon EC2 ユーザーガイド・Amazon EC2 インスタンスタイプ・AWS Compute Optimizer ユーザーガイド・EC2 Image Builder ユーザーガイド）を書いた（青い文字を押すと開く）。「」の中はドキュメントの原文どおり（日本語版は機械翻訳のため、語順が不自然なところもそのまま）。会社・台数・構成は Claude が作った例

全体像：インスタンスを起動するときに決めるもの 試験ガイド タスク 3.2



本番での問われ方：「このワークロードに最も合うインスタンスは」「データが消えないのはどれ」「通信を最も速くする置き方は」の 3 つが中心。料金の買い方（オンデマンド・リザーブド・スポットなど）は #22 で学ぶ。

この PDF の範囲外：起動テンプレートと Auto Scaling は #09、EBS のボリュームの種類は #14、セキュリティグループは #17 で学ぶ。

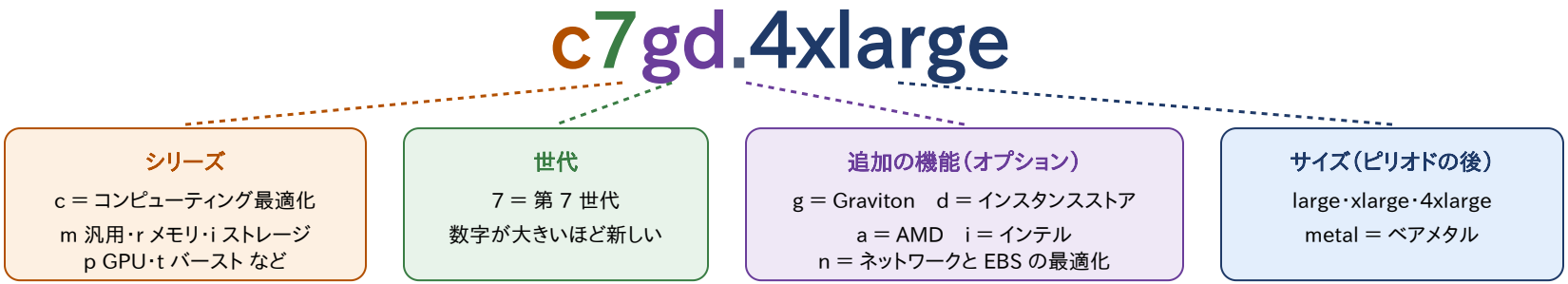
出典：Amazon EC2 の Amazon マシンイメージ（「Amazon マシンイメージ (AMI) は、Amazon EC2 インスタンスをセットアップして起動するために必要なソフトウェアを提供するイメージです。」）／Amazon EC2 とは

第1章

インスタンスタイプの選び方

名前の読み方、6 つのカテゴリ、ふだん暇なサーバーのバースト、Arm の Graviton、大きすぎ・小さすぎを見つける Compute Optimizer。

インスタンスタイプの名前を読む



読み解き（Claude が読み解いた例）：c7gd.4xlarge ＝ コンピューティング最適化・第 7 世代・Graviton・インスタンスストア付き・4xlarge。末尾に d があるタイプだけが、ローカルのインスタンスストアを持つ。

ひっかけ：オプションの g は Graviton のプロセッサ。グラフィックス用の GPU はシリーズの G（g5 など）で、位置が違う。数字はメモリの量ではなく世代。

出典：Amazon EC2 インスタンスタイプの命名規則（「インスタンスタイプは、インスタンスファミリー と インスタンスサイズ に基づいて名前が付けられます。」「2 番目のポジションは、世代 を示しています (例:7)。」）

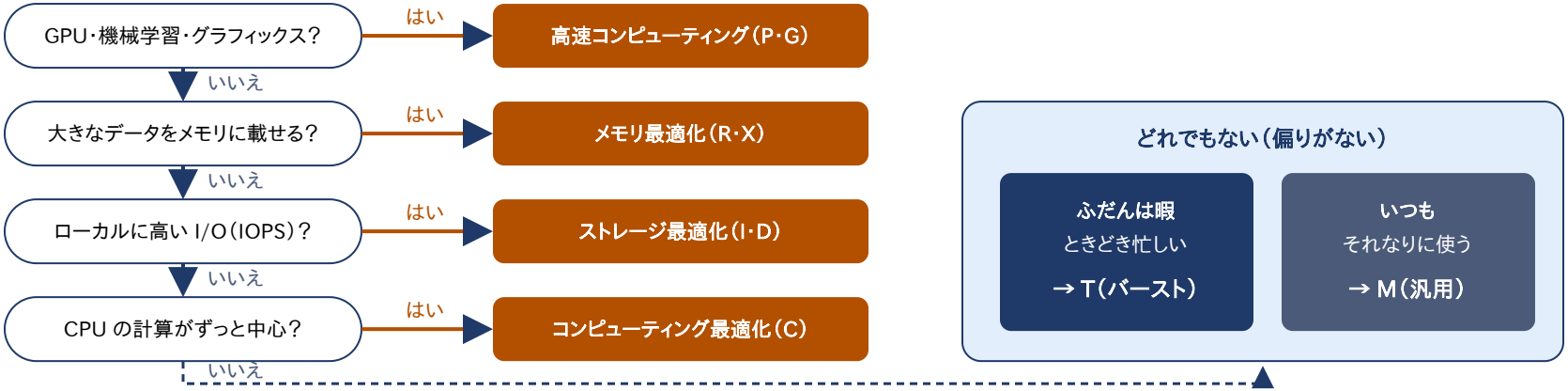
6 つのカテゴリとワークロード

カテゴリ	シリーズ	向くワークロード	選ぶ合図
汎用	M・T	Web サーバー・コードリポジトリなど、リソースを均等に使う	偏りがない・まず試す
コンピューティング最適化	C	バッチ処理・メディアのトランスコーディング・ゲームサーバー・科学モデリング	CPU の計算が中心
メモリ最適化	R・X・U・Z	インメモリ DB・大きなキャッシュ・リアルタイムのビッグデータ分析	大きなデータをメモリに
ストレージ最適化	I・D・H	NoSQL・データウェアハウス・分散ファイルシステム	ローカルへの高い I/O
高速コンピューティング	P・G・Inf・Trn・F	機械学習の学習と推論・グラフィックス・FPGA	GPU・アクセラレーター
HPC	Hpc	大規模で複雑なシミュレーション・深層学習	大規模な HPC を最高の価格性能で

Inf＝AWS Inferentia（推論）、Trn＝AWS Trainium（学習）、F＝FPGA。ストレージ最適化のデータはインスタンスストアにあり、止めると消えるので、複製する前提で使う

出典：Amazon EC2 コンピューティング最適化インスタンスの仕様（「コンピューティング最適化インスタンスは、高パフォーマンスプロセッサから恩恵を受けるコンピューティング集約型用途向けに設計されています。」）／Amazon EC2 メモリ最適化インスタンスの仕様（「メモリ最適化インスタンスは、メモリ内の大きいデータセットを処理するワークロードに対して高速なパフォーマンスを実現するように設計されています。」）／Amazon EC2 ストレージ最適化インスタンスの仕様／Amazon EC2 高速コンピューティングインスタンスの仕様／Amazon EC2 ハイパフォーマンスコンピューティングインスタンスの仕様

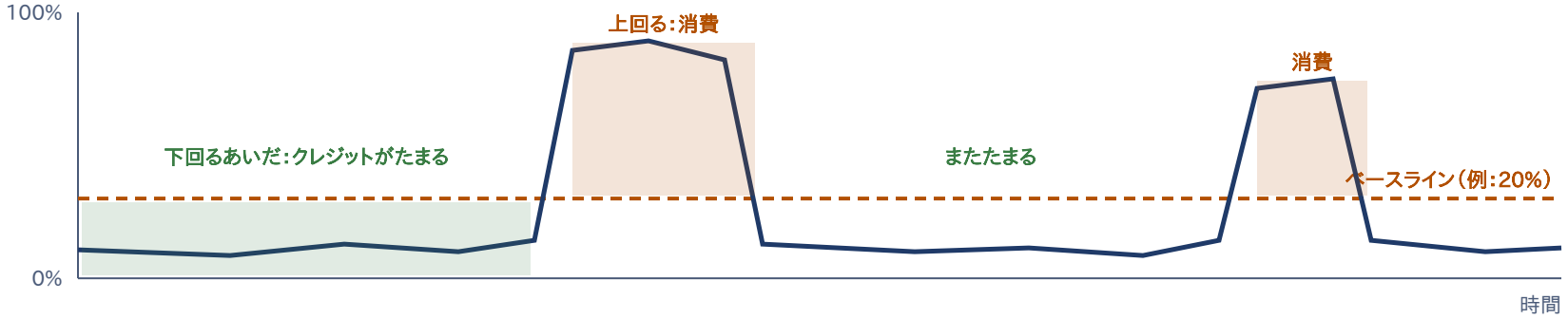
問題文から選ぶ手順



使い方：問題文の「GPU」「インメモリ」「IOPS・ローカルディスク」「トランスコーディング・バッチ」「ふだんは低い CPU」という言葉を探す。サイズで迷うなら、実際の使用率から Compute Optimizer に勧めてもらう。順番は Claude が作った覚え方で、AWS の公式の手順ではない。

出典：Amazon EC2 インスタンスタイプ（「バーストパフォーマンス (T) インスタンスは、ベースラインレベルの CPU パフォーマンスを提供しベースライン以上のバーストする機能を備えています。」）

バーストパフォーマンス（T）と CPU クレジット



向く用途：多くの汎用のワークロードは、ふだんはビジーではない。小さな Web サーバー・開発環境・マイクロサービスなど、平均は低く、ときどき跳ねる処理に合う。グラフの数字は Claude が作った例。

向かない用途：CPU をずっと高く使う処理。クレジットを使い続けることになるので、M や C などの固定パフォーマンスのインスタンスにする。

出典：バーストパフォーマンスインスタンス（「多くの汎用ワークロードでは、平均的な状態はビジーではないので、持続的で高いレベルの CPU パフォーマンスを必要としません。」）／Amazon EC2 インスタンスタイプ（「各バーストパフォーマンスインスタンスは、CPU 使用率がベースラインを下回っている間は継続的にクレジットを獲得し、ベースラインを上回っている間は継続的にクレジットを消費します。」）

無制限モードとスタンダードモード

	無制限モード（unlimited）	スタンダードモード（standard）
クレジットが尽きたら	必要なだけベースラインを超えて使い続けられる	ベースラインの性能まで下がる
追加の料金	24 時間の平均の CPU 使用率がベースラインを超えると、超えた分にかかる	かからない
既定	T4g・T3a・T3 は、デフォルトで無制限モードで起動する	設定で選ぶ
合うとき	ときどき長めに跳ねても、性能を落としたくない	料金を確実に固定したい

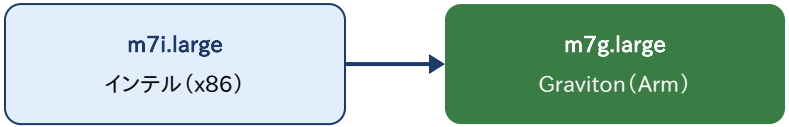
本番の場面：「T3 のインスタンスで追加料金が増えている」「CPU がいつも高い」→ 無制限モードで超えた分を払い続けている。固定パフォーマンスの M・C に変えるのが答えになりやすい。

ひっかけ：サイズを小さくすると、ベースラインも下がるので、超える分がかえって増える。スタンダードに変えると料金は止まるが、性能が下がる。

出典：バーストインスタンスの無制限モードの概念（「24 時間の平均 CPU 使用率がベースラインを超えたインスタンスには、vCPU 時間あたりの超過の使用量に対して、[均一追加料金]が発生します。」）／バーストパフォーマンスインスタンス（「T4g、T3a、および T3 インスタンスは、デフォルトで無制限モードで起動するため、起動クレジットを受け取ることはありません。」）

Graviton と Compute Optimizer

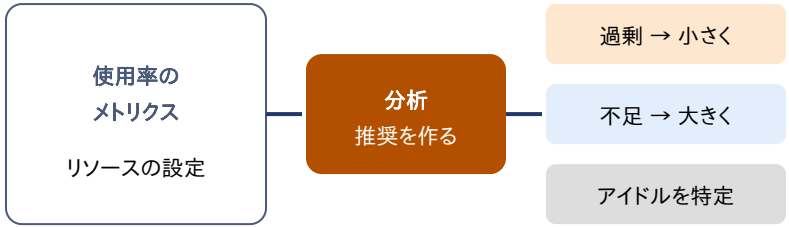
AWS Graviton（Arm ベースのプロセッサ）



移す前に：アプリ・ライブラリ・コンテナのイメージが Arm 用か
（名前の「g」が Graviton の目印）

コストパフォーマンスを上げる候補。Arm に対応していないソフトは動かない。

AWS Compute Optimizer（ライトサイジング）



「どのサイズにすべきか分からない」「大きすぎるインスタンスを見つけたい」なら Compute Optimizer。

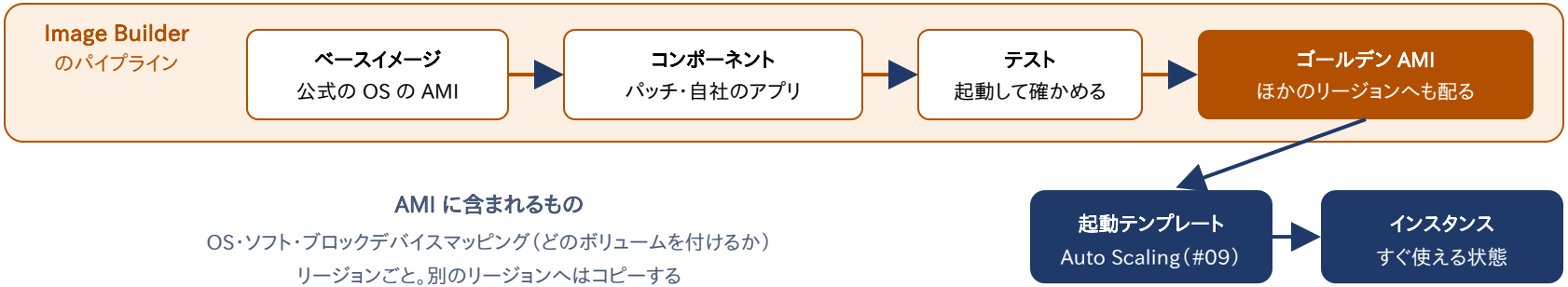
出典：バーストパフォーマンスインスタンス（「[AWS Graviton] は Amazon EC2 インスタンスで実行されるワークロードに最高のコストパフォーマンスを提供するように設計されたプロセッサファミリーです。」）／AWS Compute Optimizer とは（「AWS Compute Optimizerは、リソースの設定と使用率のAWSメトリクスを分析して、適切なサイズのレコメンデーションを提供し、アイドル状態のリソースを特定するサービスです。」）

第2章

起動と構成

何から起動するか（AMI と Image Builder）、起動時に何を渡すか（ユーザーデータ・キーペア）、中から何が読めるか（メタデータと IMDSv2）、何が消えるか（インスタンスストア・休止）、壊れたらどうなるか（自動復旧）。

AMI と EC2 Image Builder



ゴールデンイメージ：設定を済ませた AMI から起動すると、起動のたびに長いインストールをしなくてよい。スケールアウトが速くなる（ユーザーデータで毎回入れるより速い）。

手作業をなくす：「毎月パッチを当てた AMI を自動で作り直す」なら EC2 Image Builder のパイプライン。AWS Backup はデータを守るもので、パッチを当てたイメージは作らない。

出典：Amazon EC2 の Amazon マシンイメージ（「AMI を別の AWS リージョンにコピーし、そのリージョンでインスタンスを起動するために使用できます。」）／Image Builder とは（「EC2 Image Builder は、カスタマイズされ、安全で、up-to-dateサーバーイメージの作成、管理、デプロイを自動化 AWS のサービス するフルマネージド型です。」「パイプラインを設定して、所有しているイメージの更新とシステムパッチを自動化できます。」）

ユーザーデータとキーペア

ユーザーデータ（起動時のスクリプト）

```
#!/bin/bash
dnf -y install httpd
systemctl enable --now httpd
aws s3 cp s3://example-bucket/site/ \
  /var/www/html/ --recursive
```

Claude が作った例（Amazon Linux の Web サーバー）。Linux ではシェルスクリプトか cloud-init のディレクティブ。Windows では起動エージェントが処理する

いつ動く：デフォルトでは最初の起動のときだけ。再起動では動かない。

秘密は書かない：インスタンスの中から読めるが、認証や暗号化で守られていない。パスワードは Secrets Manager などに。

キーペア（接続の鍵）

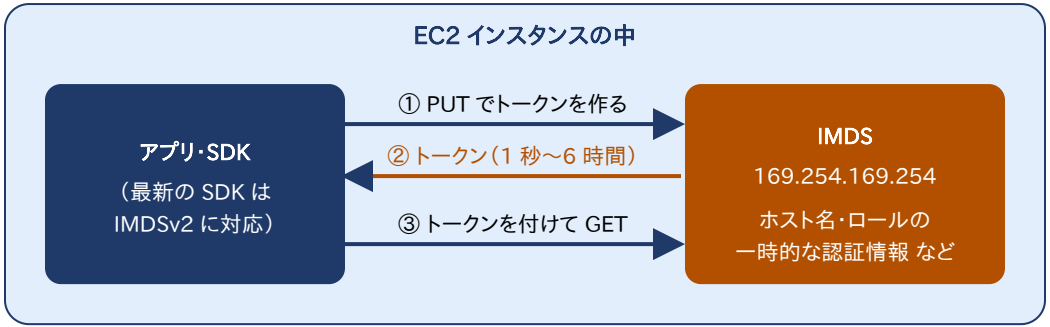
OS	プライベートキーの使い道
Linux	SSH で接続する
Windows	管理者パスワードを復号する

鍵を配らずに入る：キーペアの代わりに、Systems Manager Session Manager でブラウザや CLI から接続できる。SSH のポートを開けずに済む。

取り違え：IAM ユーザーのアクセスキーは AWS の API を呼ぶためのもので、SSH には使わない。

出典：ユーザーデータ入力を使用して EC2 インスタンスを起動するときにコマンドを実行する（「デフォルトでは、ユーザーデータスクリプトと cloud-init ディレクティブは、インスタンスの最初の起動サイクル中にのみ実行されます。」）／Amazon EC2 のキーペアと Amazon EC2 インスタンス（「Windows インスタンスの場合、管理者パスワードを復号化するにはプライベートキーが必要です。」）

インスタンスメタデータと IMDSv2



IMDSv2 を「必須」にする

トークンのない IMDSv1 は動かない

SSRF・オープンなプロキシへの多層防御

注意

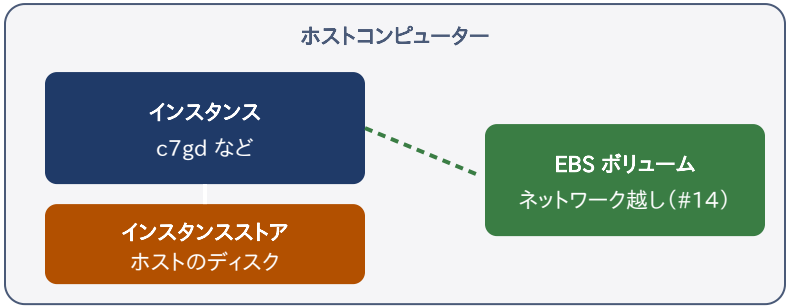
中からだけ読めるが、データは

認証・暗号化で守られていない

本番の場面：「メタデータの読み方をトークン方式だけにしたい」「SSRF でロールの認証情報を盗まれるのを防ぎたい」→ IMDSv2 を必須にする。ユーザーデータもメタデータと同じ場所から読めるので、秘密は置かない。

出典：インスタンスメタデータサービスを使用してインスタンスメタデータにアクセスする（「IMDSv2 呼び出しのみを受け入れるように各インスタンスでインスタンスメタデータサービス (IMDS) を設定できます。」）／EC2 インスタンスのインスタンスメタデータにアクセスする（「IMDSv2 が必要な場合はIMDSv1 は動作しません。」）／インスタンスメタデータを使用して EC2 インスタンスを管理する（「インスタンスメタデータおよびユーザーデータにはそのインスタンス自体内からのみアクセスできるものの、データは認証または暗号化手法によって保護されていません。」）

インスタンスストア：何をすると消えるか



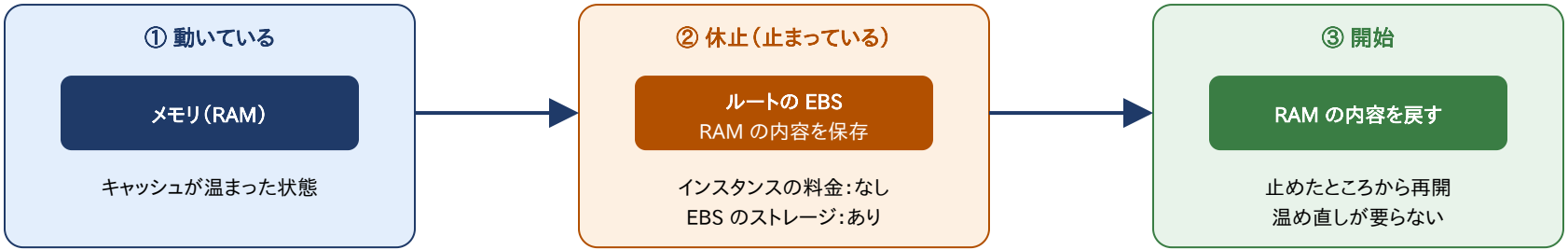
起きること	インスタンスストアのデータ	メモリの内容
再起動	残る	消える
停止 → 開始	消える	消える
休止 → 開始	消える	ルートの EBS に保存して戻る
終了	消える	消える

向く用途：バッファ・キャッシュ・スクラッチデータなど、頻繁に変わる一時的なデータ。インスタンスの料金に含まれる。

残したいデータ：EBS・S3・EFS に置くか、複数のインスタンスに複製する。止めて開始すると、ふつうは新しいホストに移るので、前のディスクには戻れない。

出典：Amazon EC2 インスタンスストアボリュームのデータの永続性（「インスタンスストアボリューム上のデータはインスタンスが再起動しても保持されます。」「ただし、インスタンスが停止、休止、終了するとデータは消滅します。」）／EC2 インスタンス用のインスタンスストアの一時ブロックストレージ（「インスタンスストアはバッファ、キャッシュ、スクラッチデータ、その他の一時的データのように頻繁に変化する情報の一時的なストレージに最適です。」）

休止：メモリの状態のまま止める



使う場面：起動してメモリにデータを読み込むまでに長い時間がかかるサーバーを、夜は止めて、翌朝その状態から再開したいとき。使うには、休止を有効にして起動し、休止の前提条件を満たす必要がある。

停止との違い：停止ではメモリの内容は消え、開始するとアプリを最初から立ち上げ直す。インスタンスストアのデータは、休止でも停止でも消える。

出典：Amazon EC2 インスタンスの休止（「休止状態に入ると、インスタンスメモリ (RAM) に置かれていた内容が、Amazon Elastic Block Store (Amazon EBS) のルートボリュームに保存されます。」「EBS ボリュームのストレージに対しては、RAM の内容のストレージも含めて、料金が発生します。」）

インスタンスの自動復旧



何のため：1 台のインスタンスの可用性を戻す。インスタンス ID や IP が変わらないので、取引先や DNS の設定を変えずに済む。復旧が失敗したら、手動で停止して開始する。

ほかとの違い：Auto Scaling は異常なインスタンスを捨てて新しい（別の ID の）インスタンスを起動する。ELB のヘルスチェックは送り先から外すだけで、インスタンスは直さない。

出典：インスタンスの自動復旧（「これらの復旧メカニズムは がシステムステータスチェックが失敗する原因となる基盤となるハードウェアまたはソフトウェアの問題AWSを検出したときに、インスタンスの可用性を復元するように設計されています。」「復元されたインスタンスは失われた要素を除いて、元のインスタンスと同じです。」「サポート対象のインスタンスを起動すると、簡易自動復旧がデフォルトで有効になります。」）

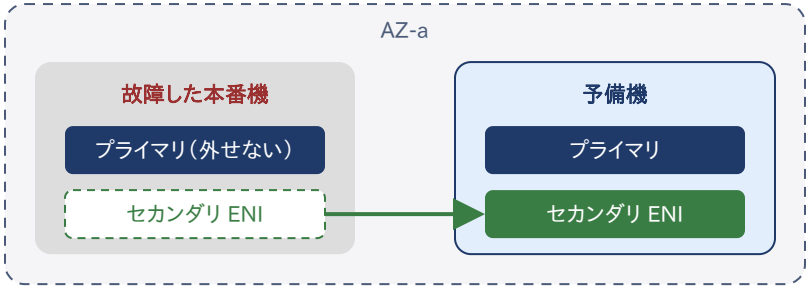
第3章

ネットワークと配置

付け替えられるネットワークカード（ENI）と固定の IP（Elastic IP）、速い通信（ENA と EFA）、ハードウェアへの置き方（クラスター・パーティション・スプレッド）。

ENI と Elastic IP：付け替えて障害を隠す

Elastic Network Interface（同じ AZ の中で付け替え）



付け替えると、プライベート IP・Elastic IP・MAC アドレス・セキュリティグループなどの属性もついていく

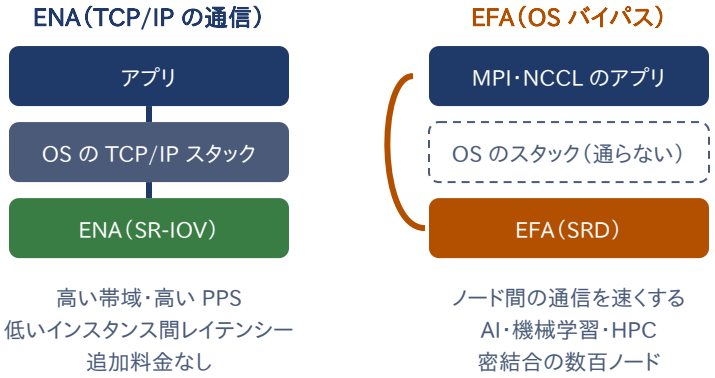
Elastic IP アドレス（静的なパブリック IPv4）

	Elastic IP	自動のパブリック IPv4
停止 → 開始	変わらない	新しくなる
別のインスタンスへ	すぐ付け替えられる	移せない
料金	使用中もアイドルも、すべてかかる	パブリック IPv4 として料金がかかる

ひっかけ：「付けていない Elastic IP は無料」は誤り。要らなければ解放する。

出典：Elastic Network Interface（「ネットワークインターフェイスを作成および設定して、同じアベイラビリティーゾーンで起動するインスタンスにアタッチできます。」「プライマリネットワークインターフェイスをインスタンスからデタッチすることはできません。」）／Elastic IP アドレス（「Elastic IP アドレスを使用すると、アドレスをアカウント内の別のインスタンスに迅速に再マッピングすることで、インスタンスやソフトウェアの障害をマスクできます。」「AWS は、実行中のインスタンスに関連付けられているパブリック IPv4 アドレスや Elastic IP アドレスを含めたすべてのパブリック IPv4 アドレスの料金を請求します。」）／インスタンスの自動復旧

拡張ネットワーキング（ENA）と EFA

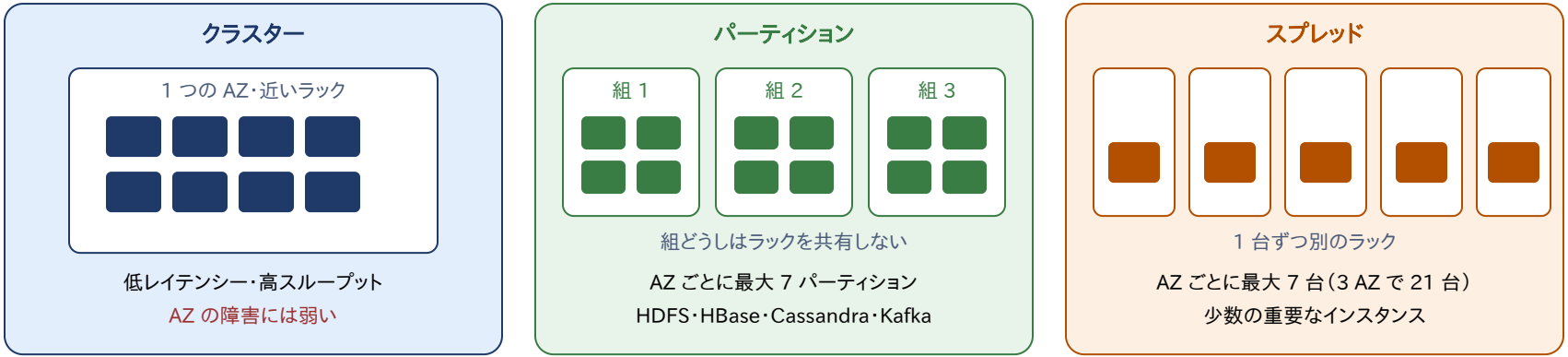


問題文の言葉	答え
ネットワークを多く使う一般のサーバー・PPS を上げたい	**ENA（拡張ネットワーキング）**
MPI・密結合の HPC・分散学習のノード間通信	**EFA+クラスタープレイスメントグループ**
固定のパブリック IP	**Elastic IP（性能は変わらない）**

ENA の範囲：Nitro ベースのインスタンスは、すべて ENA を使って拡張ネットワーキングをしている。SR-IOV は、従来の仮想化したネットワークインターフェイスより I/O の性能が高く、CPU の使用率が低い。

出典：Amazon EC2 インスタンスでの拡張ネットワーキング（「拡張ネットワーキングは、より高い帯域幅、1 秒あたりのパケット (PPS) のより高いパフォーマンス、常により低いインスタンス間レイテンシーを実現します。」「拡張ネットワーキングは追加料金なしで使用できます。」）／Amazon EC2 の AI/ML および HPC ワークロード用の Elastic Fabric Adapter（「EFA デバイスはScalable Reliable Datagram (SRD) プロトコルを介した組み込み OS バイパスや輻輳制御などの機能を提供します。」）

プレイスメントグループの 3 つの戦略



覚え方：速さ＝クラスター（まとめる）。多数を壊れにくく＝パーティション（組ごとに分ける）。少数を確実に分ける＝スプレッド（1 台ずつ分ける）。

台数で見分ける：100 台の分散システムをラックで分けたいならパーティション。スプレッドは AZ ごとに 7 台までなので足りない。

出典：プレイスメントグループのプレイスメント戦略（「低いネットワークレイテンシー、高いネットワークスループット、またはその両方からメリットを受けるアプリケーションの場合は、クラスタープレイスメントグループの使用をお勧めします。」「パーティションプレイスメントグループは、アベイラビリティーゾーンごとに最大 7 つのパーティションを持つことができます。」「スプレッドプレイスメントグループは、それぞれ異なるハードウェアに配置されるインスタンスのグループです。」）

パーティションとスプレッドをくわしく

	パーティション	スプレッド（ラックレベル）
分ける単位	インスタンスの組（パーティション）ごとに別のラック	インスタンス 1 台ずつ別のハードウェア
上限	AZ ごとに最大 7 パーティション（台数は多くてよい）	AZ ごとに実行中のインスタンスは最大 7 台
障害の影響	1 つのハードウェアの障害は、そのパーティションの中だけ	1 台のハードウェアの障害は、その 1 台だけ
アプリへの情報	どのパーティションかを、トポロジー対応のアプリと共有できる	—
合う用途	HDFS・HBase・Cassandra などの大規模な分散・複製	互いに分けておきたい少数の重要なインスタンス

計算の例（AWS のドキュメントの例）：3 つの AZ があるリージョンでは、AZ ごとに 7 台 × 3 ＝ 合計 21 台までスプレッドプレイスメントグループで動かせる。

出典：プレイスメントグループのプレイスメント戦略（「各パーティション内のインスタンスは、他のパーティション内のラックを共有しないため、単一のハードウェア障害の影響は関連付けられたパーティションのみに留まります。」「この情報は、HDFS、HBase、Cassandra などトポロジー対応アプリケーションと共有できます。」「例えば、3 つのアベイラビリティーゾーンがあるリージョンでは、アベイラビリティーゾーンごとにインスタンス 7 個として、合計 21 個のインスタンスをスプレッドプレイスメントグループ内で実行できます。」）

まとめ

インスタンスタイプ
名前=シリーズ・世代・オプション・サイズ。C 計算、R メモリ、I ローカル I/O、P・G GPU、T ふだん暇

起動と構成
AMI と Image Builder。ユーザーデータは最初の起動だけ・秘密は書かない。IMDSv2 を必須に

消えるもの・戻すもの
インスタンスストアは停止・休止・終了で消える。休止は RAM を EBS に。自動復旧は同じ ID と IP

ネットワークと配置
ENI は同じ AZ で付け替え。Elastic IP は常に有料。ENA と EFA。クラスター・パーティション・スプレッド

ひっかけ	正しくは
CPU がいつも高いのに T3	**M・C に変える**
インスタンスストアは停止でも残る	**再起動だけ残る**
メモリの状態を残すのに停止	**休止**
プライマリの ENI を付け替え	**セカンダリの ENI**
付けていない Elastic IP は無料	**常に料金がかかる**
100 台をスプレッドで	**パーティション**

次の #13 では、Amazon S3 を学ぶ。

出典:このPDFの各ページの資料